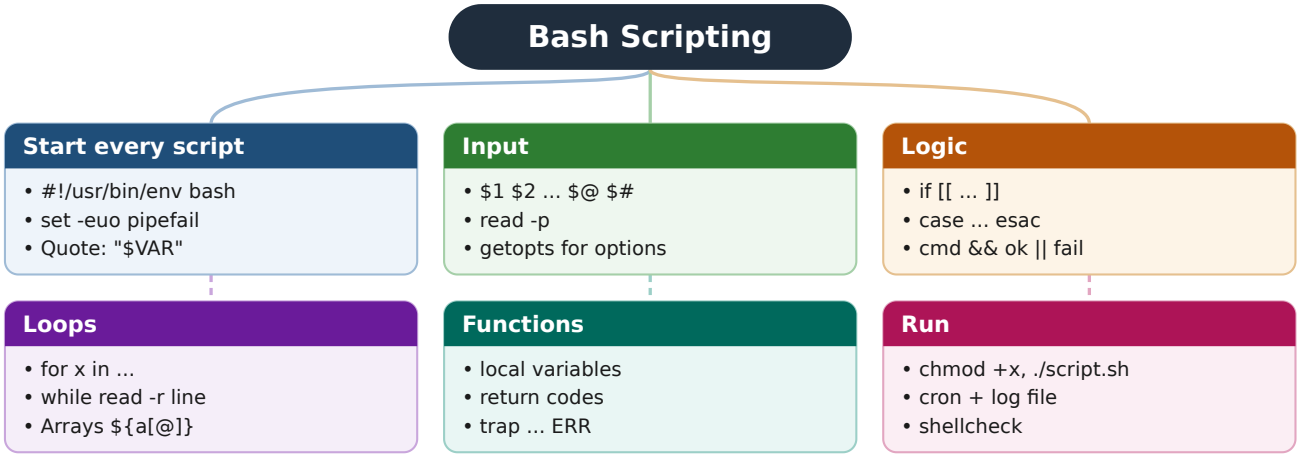
5. Bash Scripting

In this section: 5.1 Script basics · 5.2 Conditions · 5.3 Loops and case · 5.4 Functions, error handling, logging · 5.5 Example: disk usage alert · 5.6 More Bash features · 5.7 Bash syntax: what is what



5.1 Script basics

```
#!/usr/bin/env bash        # shebang: which program runs the script
set -euo pipefail          # stop on errors, unset variables, and failed pipes

NAME="server1"             # variable (no spaces around =)
echo "Checking $NAME"
echo "Today: $(date +%F)"  # command substitution
```

Run it: `chmod +x script.sh` then `./script.sh`.

Concept	Syntax
Variable	`VAR=value`, use as `"$VAR"` (always quote)
Arguments	`$1`, `$2` ... ; `$#` = count ; `$@` = all ; `$0` = script name
Exit code of last command	`$?` (0 = success, anything else = failure)
Read input	`read -p "Name: " NAME`
Single vs double quotes	`'$VAR'` = literal text, `"$VAR"` = value
Arithmetic	`$((a + b))`

5.2 Conditions

```
if [[ -f "/etc/nginx/nginx.conf" ]]; then
  echo "Config exists"
elif [[ "$ENV" == "prod" ]]; then
  echo "Production"
else
  echo "Missing"
fi
```

Test	True when	Test	True when
`-f file`	File exists	`-z "$s"`	String is empty
`-d dir`	Directory exists	`-n "$s"`	String is not empty
`-x file`	File is executable	`"$a" == "$b"`	Strings are equal
`$a -eq $b`	Numbers equal	`-gt`, `-lt`, `-ge`, `-le`	Greater / less than

Short form: `cmd && echo ok || echo failed`.

5.3 Loops and case

```
for server in web1 web2 web3; do
  ping -c 1 "$server" > /dev/null && echo "$server UP" || echo "$server DOWN"
done

while read -r line; do
  echo "Line: $line"
done < servers.txt

case "$1" in
  start) echo "Starting" ;;
  stop)  echo "Stopping" ;;
  *)     echo "Usage: $0 {start|stop}"; exit 1 ;;
esac
```

5.4 Functions, error handling, logging

```
log() { echo "$(date '+%F %T') [$1] $2"; }

check_service() {
  local svc="$1"
  if systemctl is-active --quiet "$svc"; then
    log INFO "$svc is running"
  else
    log ERROR "$svc is down"
    return 1
  fi
}

trap 'log ERROR "Script failed on line $LINENO"' ERR
check_service nginx
```

5.5 Example: disk usage alert

```
#!/usr/bin/env bash
set -euo pipefail
THRESHOLD=80
USAGE=$(df / --output=pcent | tail -1 | tr -dc '0-9')
if (( USAGE > THRESHOLD )); then
  echo "WARNING: root disk at ${USAGE}%"
  exit 1
fi
echo "OK: root disk at ${USAGE}%"
```

Schedule it with cron: `*/10 * * * * /opt/scripts/disk_check.sh >> /var/log/disk_check.log 2>&1`.

Best practices: quote variables, use `set -euo pipefail`, check inputs, log clearly, use exit codes, check scripts with `shellcheck`, no secrets inside scripts.

5.6 More Bash features

```
servers=("web1" "web2" "db1")            # array
echo "${servers[0]}"                     # first item
echo "${#servers[@]}"                    # number of items
for s in "${servers[@]}"; do echo "$s"; done

name="${1:-default}"                     # default value if $1 is empty
file="report.tar.gz"; echo "${file%.tar.gz}"   # strip a suffix → report

cat <<EOF > /tmp/app.conf                # here-document: write multi-line text
port=8080
env=dev
EOF

while getopts "e:v" opt; do              # parse options: -e <env> -v
  case $opt in
    e) ENV="$OPTARG" ;;
    v) VERBOSE=1 ;;
    *) echo "Usage: $0 -e env [-v]"; exit 1 ;;
  esac
done
```

5.7 Bash syntax: what is what

Annotated script (every line explained in the comments):

```
#!/usr/bin/env bash
# ^ SHEBANG: first line, tells Linux to run this file with bash
# Lines starting with # are COMMENTS (ignored)

set -euo pipefail
# -e  stop if any command fails
# -u  stop if an unset variable is used
# -o pipefail  a pipeline fails if any part of it fails

APP_NAME="web"                 # VARIABLE: NAME=value  (no spaces around =)
MAX_RETRIES=3                  # numbers are stored as text, too
LOG_FILE="/tmp/${APP_NAME}.log"  # ${VAR} = use the value, braces mark where the name ends

TODAY=$(date +%F)              # COMMAND SUBSTITUTION: $(command) = its output
COUNT=$(( MAX_RETRIES * 2 ))   # ARITHMETIC: $(( ... ))

log() {                        # FUNCTION definition: name() { ... }
  local level="$1"             # local = variable only inside the function; $1 = first argument
  shift                        # drop $1, so "$*" is the rest of the arguments
  echo "$(date '+%T') [$level] $*" | tee -a "$LOG_FILE"   # | PIPE output into tee; -a = append
}

if [[ $# -lt 1 ]]; then        # IF: [[ test ]]; $# = number of script arguments
  echo "Usage: $0 <url>" >&2   # $0 = script name; >&2 = print to stderr (errors)
  exit 1                       # EXIT CODE: 0 = success, non-zero = failure
fi

URL="$1"
for i in $(seq 1 "$MAX_RETRIES"); do           # FOR loop over 1..3
  # the command itself is the test; > /dev/null = discard its output
  if curl -fsS --max-time 5 "$URL" > /dev/null; then
    log INFO "$URL is up (attempt $i, $TODAY)"
    exit 0
  fi
  log WARN "attempt $i failed, retrying"
  sleep 2
done

log ERROR "$URL is down after $MAX_RETRIES attempts (counter $COUNT)"
exit 2
```

Element	Syntax	Meaning
Variable	`NAME=value` · use `"$NAME"`	No spaces around `=`. Always quote when using
Single quotes	`'$HOME'`	Literal text, nothing expanded
Double quotes	`"$HOME"`	Variables expanded, spaces kept safe
Command substitution	`$(cmd)`	Replaced by the command's output
Arithmetic	`$(( a + b ))`	Integer math: `+ - * / %`
Special variables	`$0` `$1..$9` `$#` `$@` `$?` `$$`	Script name, arguments, count, all args, last exit code, PID
Test	`[[ ... ]]`	Conditions (file, string, number)
Command as condition	`if grep -q x file; then`	Uses the command's exit code
And / or	`cmd1 && cmd2` · `cmd1 || cmd2`	Run cmd2 only if cmd1 succeeds / fails
Redirect	`>` `>>` `2>` `&>` `< file`	Overwrite, append, errors, both, input

Element	Syntax	Meaning
Pipe	`cmd1 | cmd2`	Output of cmd1 becomes input of cmd2
Discard	`> /dev/null 2>&1`	Throw away output and errors
Function	`name() { ...; }`	Call as `name arg1 arg2`; arguments are `$1`, `$2` inside
Block endings	`if…fi` · `case…esac` · `for/while… do…done`	Bash closes blocks with reversed words
Background	`cmd &`	Run without waiting

Common mistakes: spaces around `=` (`A = 1` fails) · unquoted variables with spaces · missing `then`/`do` · forgetting `chmod +x` · Windows line endings (`\r`) in scripts edited on Windows (fix: `sed -i 's/\r$//' script.sh`).

REMEMBER

- Exit code **0 = success**, anything else = failure (`$?`).
- Tests: `-f` file · `-d` directory · `-z` empty · `-eq/-gt/-lt` numbers · `==` strings.
- Never put passwords in scripts: read them from the environment.